

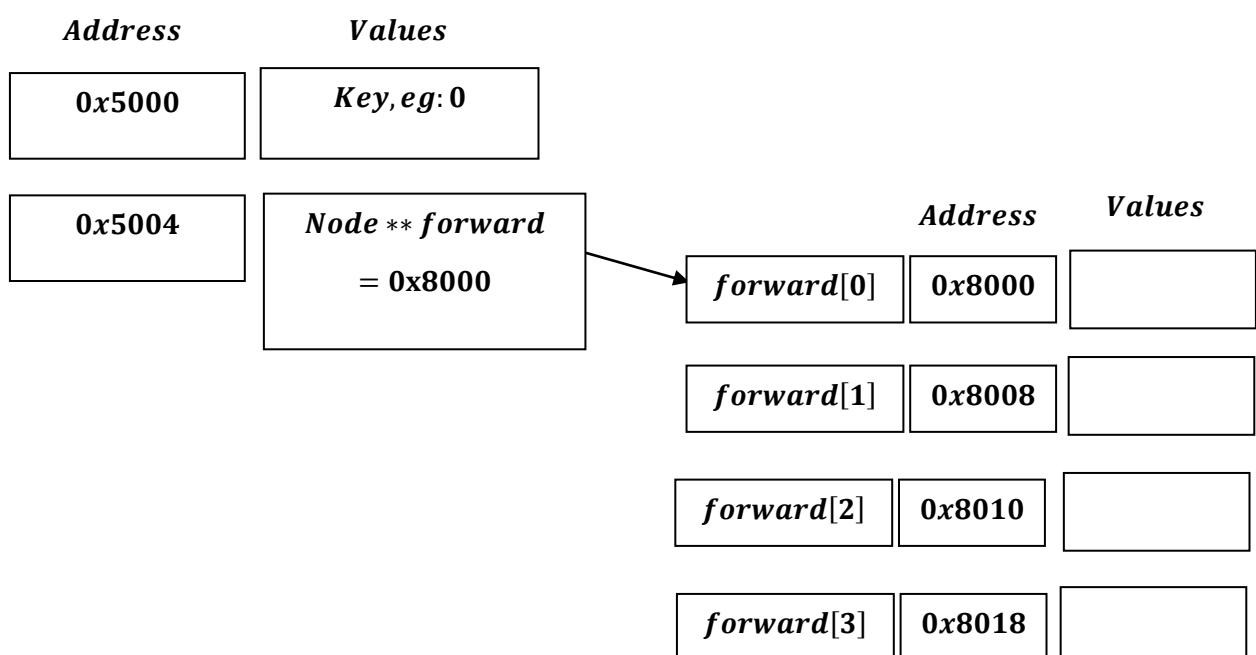
Skip List – Memory Referencing Overview

```
struct Node {  
    int key;  
    // Array of pointers to next nodes, one for each level  
    struct Node **forward;  
};  
struct Node *n = createNode(key, rlevel);
```

Formation of Struct Node – An Memory Layout Example

```
n->key = key;  
  
    // Allocate memory for the forward pointer array (level + 1  
elements)  
    n->forward = (struct Node **)malloc(sizeof(struct Node *) * (level  
+ 1));
```

*Struct * Node*



→ $baseaddress + offset = 0x5000 + 0 [bytes] = \text{Where key address is stored.}$

→ $baseaddress + offset = 0x5000 + 4 [bytes] = ** forward, where$
 $forward[0]'s address is stored.$

The Compiler's "Secret" Calculation

When the compiler sees `node->key = 0;`, it thinks:

- "Get the base memory address stored in the pointer node." (e.g., `0x5000`)
- "Look up the definition of the Node struct. The key member is at the beginning, so its offset is 0."
- "Calculate the final address: $base\ address + offset \Rightarrow 0x5000 + 0 \Rightarrow 0x5000.$ "
- "Write the value `0` to the memory location starting at `0x5000`."

When the compiler sees an assignment to the pointer itself,

such as `node->forward = 0x8000;` (where `0x8000` is the memory address of the newly allocated dynamic array), it uses that exact same logical sequence!

- Get the base memory address stored in the pointer `node`." (e.g., `0x5000`)
- Look up the definition of the Node struct. The forward pointer comes directly after the integer key variable (which is 4 bytes long). So, the offset for forward is 4."
- "Calculate the final address:

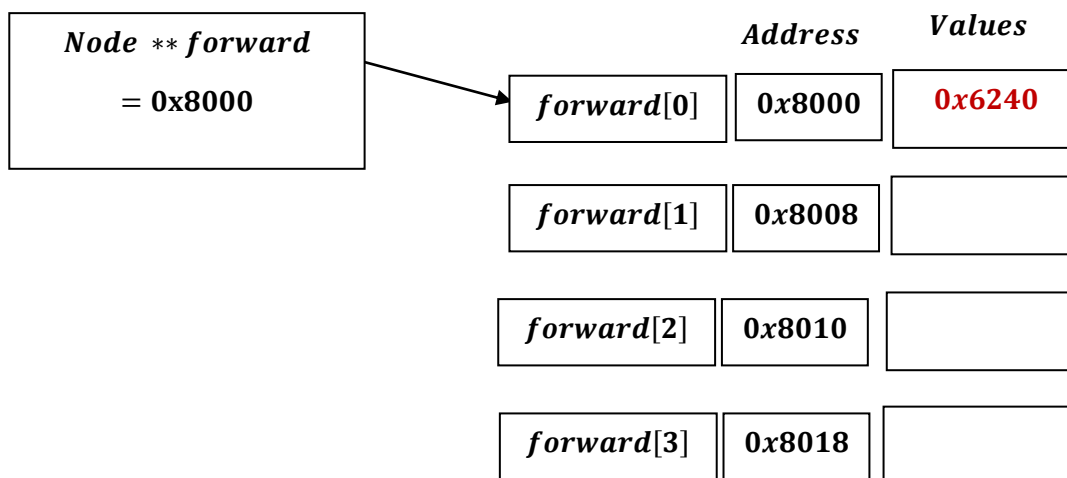
$base\ address + offset \Rightarrow 0x5000 + 4 \Rightarrow 0x5004.$

"Write the value `0x8000` (the 8-byte pointer address) to the memory location starting at `0x5004`."

The "Double Jump" (When we add the brackets [])

It is important to note that writing to `node->forward` is just setting up the map. If the compiler sees `node->forward[0] = 0x6240;`, it has to do a two-step operation:

1. Step 1 (The Struct Offset): It does the exact calculation above ($0x5000 + 4 = 0x5004$) to read the value stored there, which tells it the array lives at `0x8000`.
2. Step 2 (The Array Offset): It takes that new base address (`0x8000`), looks at the index `[0]`, calculates the array offset ($0 * 8 \text{ bytes} = 0$), and finally writes the target address (`0x6240`) into `0x8000`.



Now, as :

```
struct Node **forward;
```

And ,when we write:

```
n->forward = (struct Node **)malloc(sizeof(struct Node *) * (level + 1));
```

This means:

if $level = 3$, then $level + 1 = 4$ and Node pointer variable of 8 bytes in 64 bit architecture.

Then it translates to:

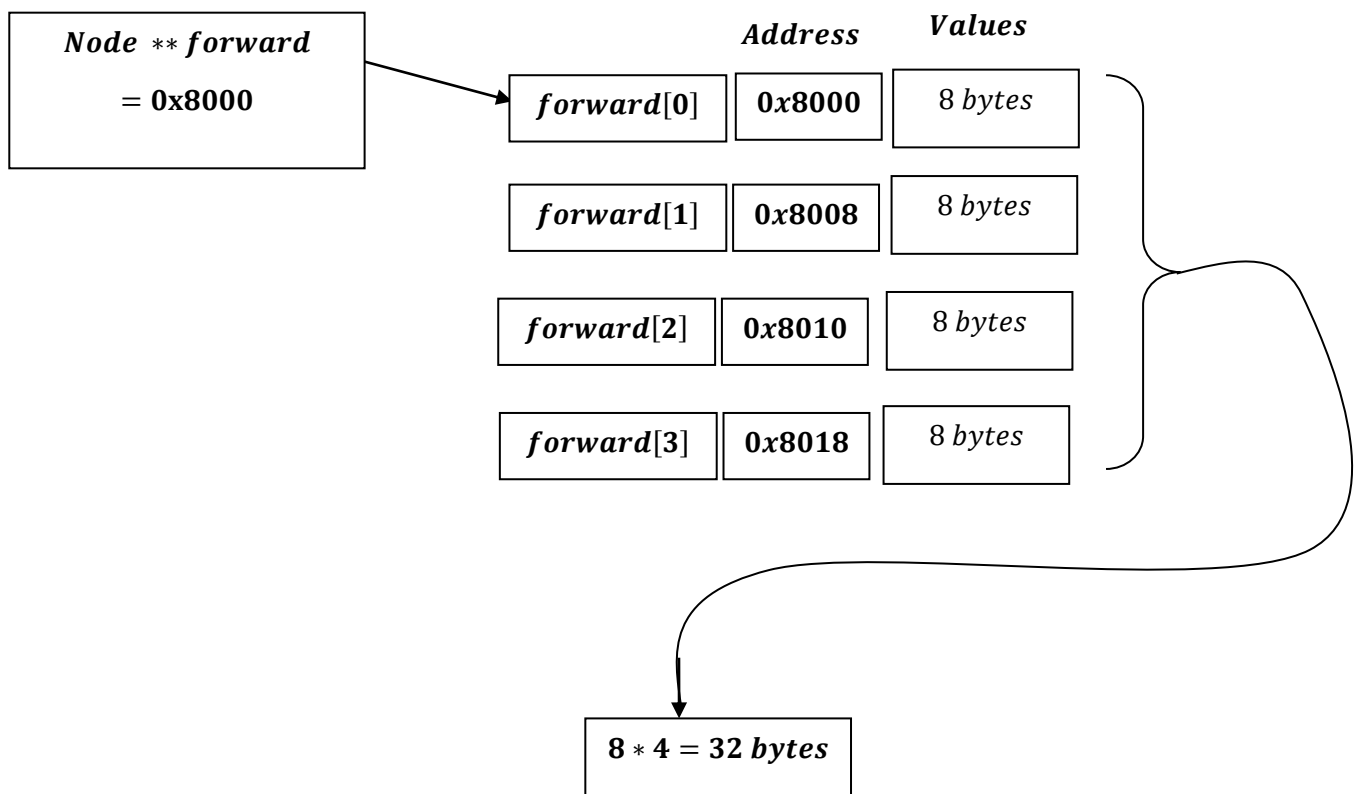
*$n \rightarrow forward = (struct Node **)malloc(size\ of(8\ bytes) * (4));$*

We are telling the compiler: "Take these raw 32 bytes (8×4) that malloc just created, and label them as a block that will hold pointers."

*As we are assigning, $n \rightarrow forward = (struct Node **)malloc(size\ of(8\ bytes) * (4));$*

hence, $n \rightarrow forward$ will hold the first address of the memory block i.e.

$forward[0]$.



Now when we, write: $n \rightarrow forward[0]$, compiler translates it into:

$(n \rightarrow forward + 0)$.*

Similarly:

$n \rightarrow \text{forward}[1]$, compiler translates it into : $*(n \rightarrow \text{forward} + 1)$

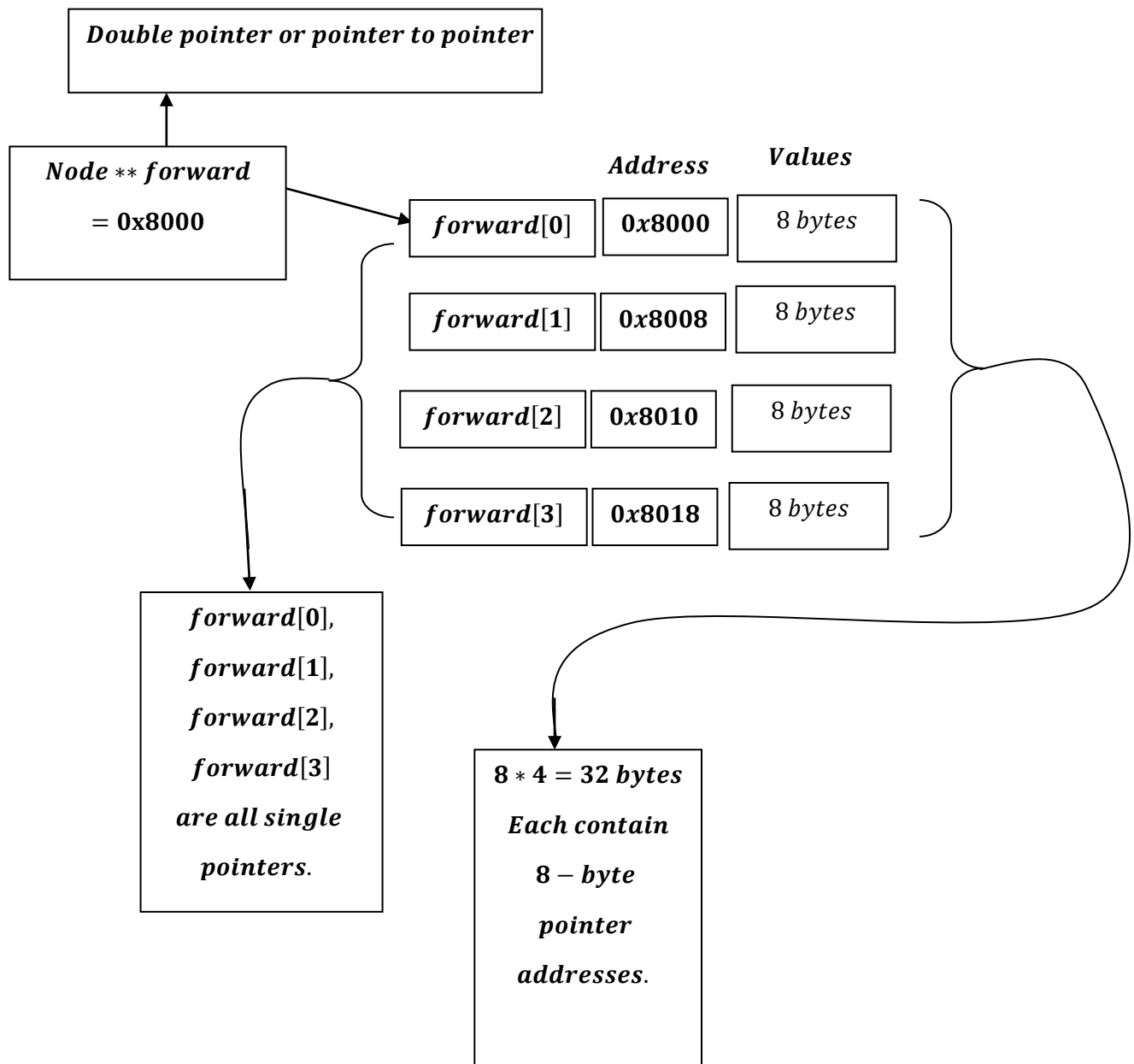
$n \rightarrow \text{forward}[2]$, compiler translates it into : $*(n \rightarrow \text{forward} + 2)$

$n \rightarrow \text{forward}[3]$, compiler translates it into : $*(n \rightarrow \text{forward} + 3)$

When the compiler evaluates the $+ 0$ inside $(n \rightarrow \text{forward} + 0)$:

→ It asks: "**What type of data does this base address point to?**"

- Because forward is a $\text{Node} **$ (pointer to a pointer), the compiler knows it points to an 8 – byte pointer.



$$\text{Formula} = \text{Base Address} + (\text{Index} \times \text{Size in Bytes})$$

Note:

$n \rightarrow \text{forward}: 0x8000$

$(n \rightarrow \text{forward} + 0)$ translates as: $0x8000 + 0 \times 8 \text{ bytes} := 0x8000$, where value(8 bytes address) at $n \rightarrow \text{forward}[0]$ is stored. $[0_{10}(\text{Decimal}) = 0_{16}(\text{Hex})]$

$(n \rightarrow \text{forward} + 1)$ translates as: $0x8000 + 1 \times 8 \text{ bytes} := 0x8008$, where value(8 bytes address) at $n \rightarrow \text{forward}[1]$ is stored. $[8_{10}(\text{Decimal}) = 8_{16}(\text{Hex})]$

$(n \rightarrow \text{forward} + 2)$ translates as: $0x8000 + 2 \times 8 \text{ bytes} := 0x8010$, where value(8 bytes address) at $n \rightarrow \text{forward}[2]$ is stored. $[16_{10}(\text{Decimal}) = 10_{16}(\text{Hex})]$

$(n \rightarrow \text{forward} + 3)$ translates as: $0x8000 + 3 \times 8 \text{ bytes} := 0x8018$, where value(8 bytes address) at $n \rightarrow \text{forward}[3]$ is stored. $[24_{10}(\text{Decimal}) = 18_{16}(\text{Hex})]$

```
struct SkipList {  
    int level;           // Current maximum level in the list  
    struct Node *header; // Pointer to the header node  
};
```

And, we have :

```
lst->header = createNode(-1, MAX_LEVEL);
```

And After call, we have: –

```
struct Node *createNode(int key, int level) {  
    // Allocate memory for the node  
    struct Node *n = (struct Node *)malloc(sizeof(struct Node));  
    n->key = key;
```

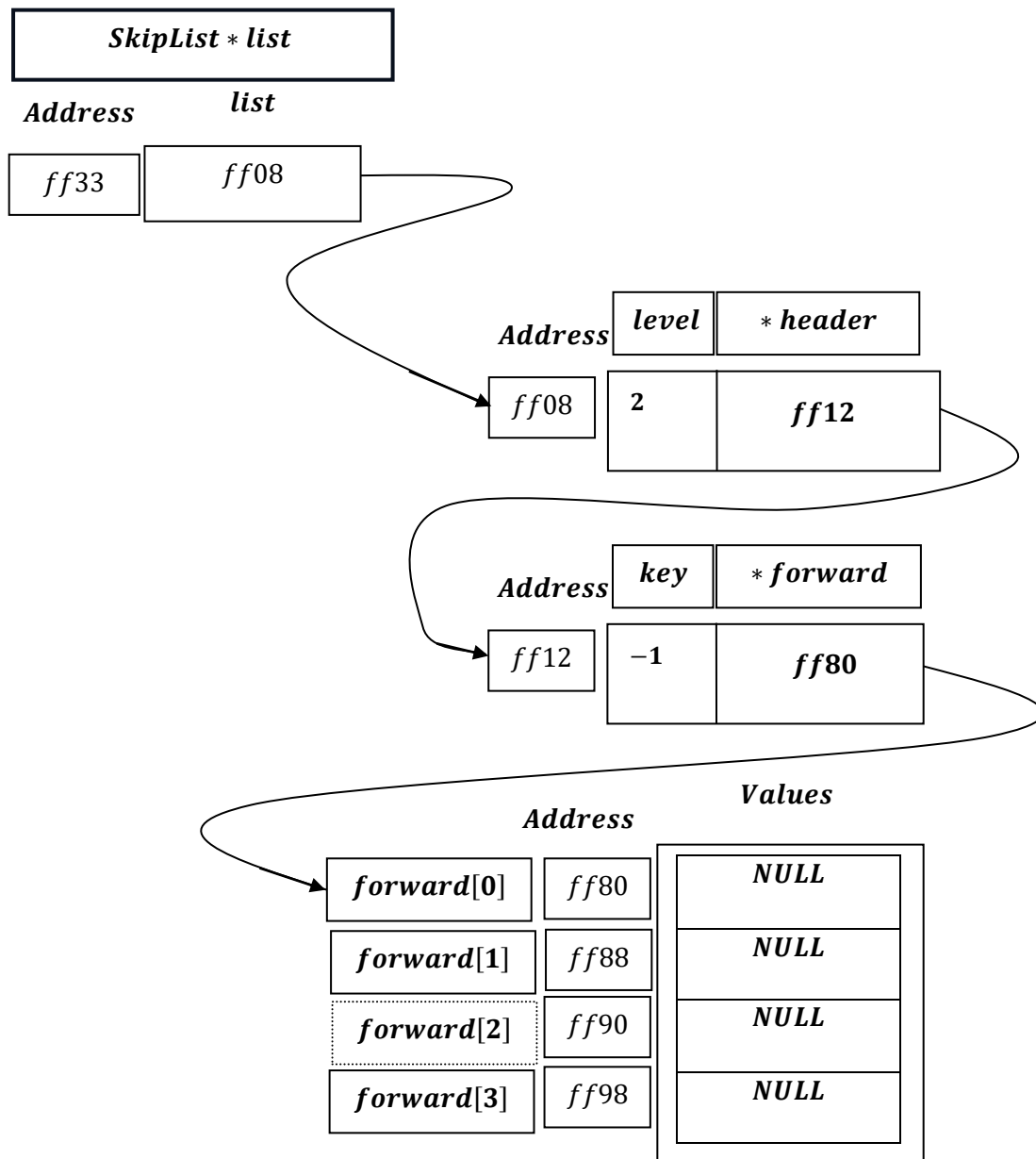
```

    // Allocate memory for the forward pointer array (level + 1
elements)
    n->forward = (struct Node **)malloc(sizeof(struct Node *) * (level
+ 1));

    // Initialize pointers to NULL
    for (int i = 0; i <= level; i++) {
        n->forward[i] = NULL;
    }
    return n;
}

```

Therefore, we have seen diagrammatically:



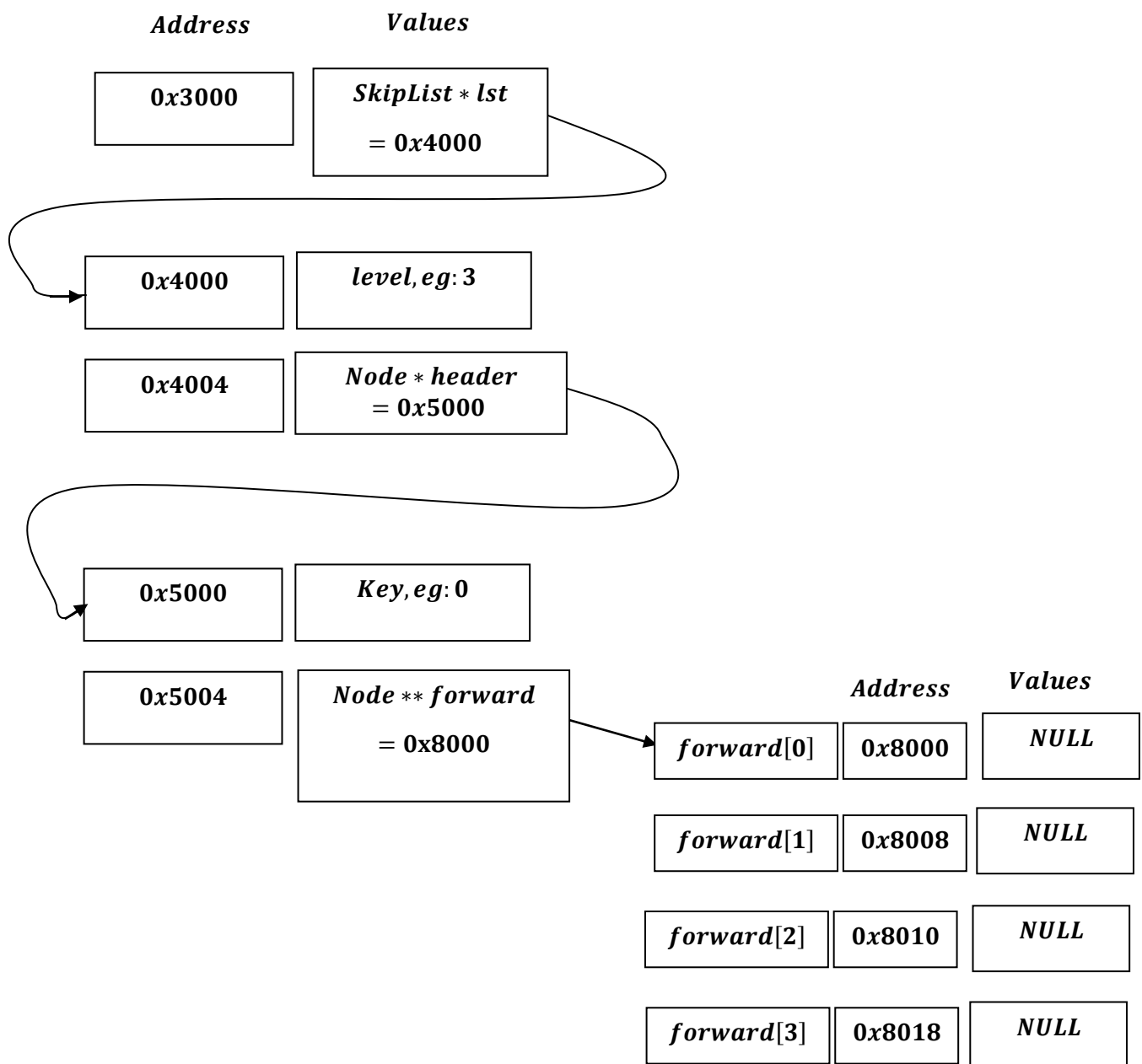
But its actually looks like *[Lets change the addresses]*:

```
struct SkipList {  
    int level;           // Current maximum level in the list  
    struct Node *header; // Pointer to the header node  
};
```

*So, compiler will first allocate : $Base(0x4000) + offset(0) = 0x4000$
for member integer variable **Level** and as it is integer ,
it will allocate 4 bytes for it.*

*And then it will allocate for $Node * header$ at address : $Base(0x4000) + offset(4)$
 $= 0x4004$ for member pointer variable : header.*

Therefore ,it will look like :



SkipList's pointer variable ** lst* will keep on pointing at 0x4000 *Fixed
At
Level*

even during next ** header* address is calculated i.e. Base Address + Offset [0x4000 + 4 = 0x4004].

*i. e. to move next address just + 4 offset is address but **SkipList * lst** will keep on pointing fixed at **LEVEL's address[0x4000]** and doesn't point to **Node * header**.*

Similarly,

***Node's** pointer variable * **header** will keep on pointing at **0x5000** $\left[\begin{array}{c} \text{Fixed} \\ \text{At} \\ \text{Key} \end{array} \right]$*

*even during next ** **forward** address is calculated i. e. Base Address + Offset [0x5000 + 4 = 0x5004].*

*i. e. to move next address just + 4 offset is address but **Node * header** will keep on pointing fixed at **Key's Address[0x5000]** and doesn't point to **Node * header**.*

*Same for node's double pointer variable ** **Forward** , which will keep pointing at node $\rightarrow$ forward[0]'s address fixed : 0x8000 and approaching next addresses of node $\rightarrow$ forward[1] , node $\rightarrow$ forward[2] , node $\rightarrow$ forward[3] even , node $\rightarrow$ forward[0] , only math is used :*

** (**n $\rightarrow$ forward + 0**) translates as: $0x8000 + 0 \times 8 \text{ bytes} := 0x8000$, where value(8 bytes address) at n $\rightarrow$ forward[0] is stored. $[0_{10}(\text{Decimal}) = 0_{16}(\text{Hex})]$*

** (**n $\rightarrow$ forward + 1**) translates as: $0x8000 + 1 \times 8 \text{ bytes} := 0x8008$, where value(8 bytes address) at n $\rightarrow$ forward[1] is stored. $[8_{10}(\text{Decimal}) = 8_{16}(\text{Hex})]$*

** (**n $\rightarrow$ forward + 2**) translates as: $0x8000 + 2 \times 8 \text{ bytes} := 0x8010$, where value(8 bytes address) at n $\rightarrow$ forward[2] is stored. $[16_{10}(\text{Decimal}) = 10_{16}(\text{Hex})]$*

** (**n $\rightarrow$ forward + 3**) translates as: $0x8000 + 3 \times 8 \text{ bytes} := 0x8018$, where value(8 bytes address) at n $\rightarrow$ forward[3] is stored. $[24_{10}(\text{Decimal}) = 18_{16}(\text{Hex})]$*

Similarly for **update**:

```
struct Node *update[MAX_LEVEL + 1];
```

update[3 + 1 = 4] ,

MAX_LEVEL evaluates to 3. Consequently, the array size calculation is update[3 + 1 = 4].

This means the array is allocated to hold exactly 4 elements.

Before memset call, the update array may look like :

Array Element	Hypothetical Stack Address	Value (Garbage Data)	Pointer Status
update[0]	0x7fff0000 to 0x7fff0007	0x4a3f8921bce45099	Wild Pointer (Points to a random, invalid address)
update[1]	0x7fff0008 to 0x7fff000f	0x0000000000000002	Wild Pointer
update[2]	0x7fff0010 to 0x7fff0017	0x7ffee1245a00881c	Wild Pointer
update[3]	0x7fff0018 to 0x7fff001f	0x0000000000000000	Accidentally NULL (Just by chance)

More simpler way:

	Address	Values	
update[0]	0x7fff0000	0x4a3f8921bce45099	[8 bytes]
update[1]	0x7fff0008	0x0000000000000002	[8 bytes]
update[2]	0x7fff0010	0x7ffee1245a00881c	[8 bytes]
update[3]	0x7fff0018	0x0000000000000000	[8 bytes]

```
memset(update, 0, sizeof(struct Node *) * (MAX_LEVEL + 1));
```

“memset” stands for Memory Set. It takes a block of memory and rapidly fills it with a specific value. Here ‘0’ or NULL.

	Address	Values
update[0]	0x7fff0000	NULL
update[1]	0x7fff0008	NULL
update[2]	0x7fff0010	NULL
update[3]	0x7fff0018	NULL

And how compiler will translate :

→ $update[0] = * (update + 0)$

→ $update[1] = * (update + 1)$

→ $update[2] = * (update + 2)$

→ $update[3] = * (update + 3)$

And how data will get stored:

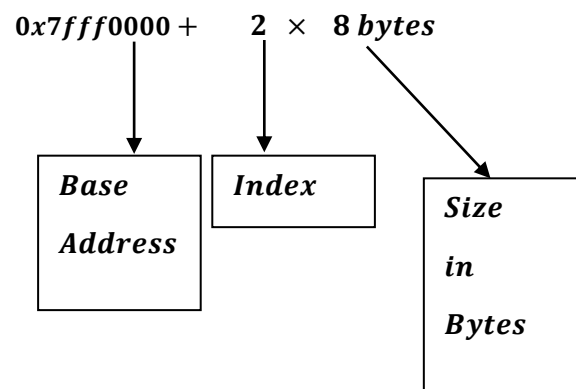
** (update + 0) translates as: $0x7fff0000 + 0 \times 8 \text{ bytes} := 0x7fff0000$, where value(8 bytes address) at **update[0]** is stored. [$0_{10}(\text{Decimal}) = 0_{16}(\text{Hex})$]*

** (update + 1) translates as: $0x7fff0000 + 1 \times 8 \text{ bytes} := 0x7fff0008$, where value(8 bytes address) at **update[1]** is stored. [$8_{10}(\text{Decimal}) = 8_{16}(\text{Hex})$]*

** (**update + 2**) translates as: $0x7fff0000 + 2 \times 8 \text{ bytes} := 0x7fff0010$, where value(8 bytes address) at **update[2]** is stored. [$16_{10}(\text{Decimal}) = 10_{16}(\text{Hex})$]*

** (**n → forward + 3**) translates as: $0x7fff0000 + 3 \times 8 \text{ bytes} := 0x7fff0018$, where value(8 bytes address) at **update[3]** is stored. [$24_{10}(\text{Decimal}) = 18_{16}(\text{Hex})$]*

And



```
struct Node *update[MAX_LEVEL + 1];
```

*Node * update is single pointer variable and it will allocate 8 bytes $\times$ (level + 1), if level = 3, then: $8 \times 4 = 32$ bytes of memory and why 8 bytes as * update is a local temporary variable created for rewiring needed for operation like: **insertion** and **deletion** which will hold 8 byte addresses.*

Now, we have got an overview of memory referencing but Why 8 byte addresses? For pointer variables and why 4 bytes addresses for integer variable? The true memory allocation will be discussed in next document.
